

LAPORAN PRAKTIKUM

TUGAS – Praktik 3

(TEKNOLOGI IOT II)



Disusun Oleh :

Nama : Zahara Ummu Abdillah
Kelas : 3B
NPM :25782058

Dosen :

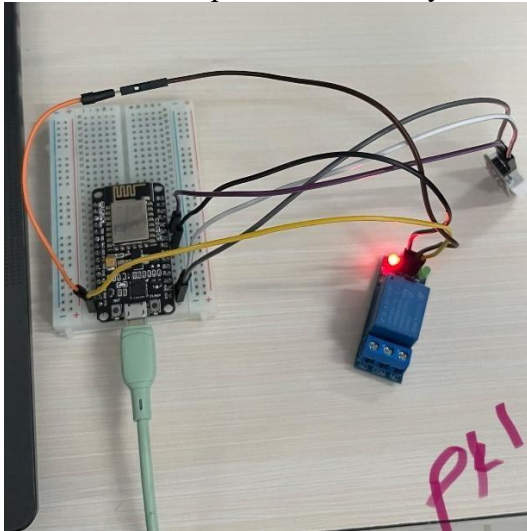
1. Dr. Ir. Septafiansyah Dwi Putra
2. Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG 2026**

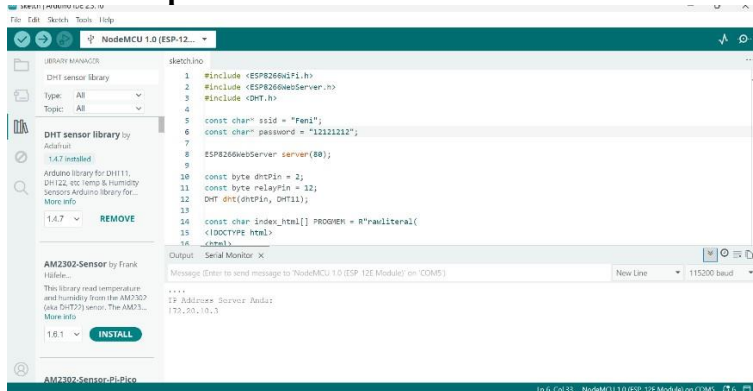
➤ HASIL PELAKSANAAN PRAKTIKUM

3.5.1 Praktikum 1: NodeMCU sebagai Web Server (Menyajikan HTML)

Tujuan: Menjadikan NodeMCU sebagai Penyedia Layanan (Server) yang memiliki alamat IP sendiri dan bisa menyajikan tampilan halaman web (dashboard) ke pengguna. Berikut hasil implementasi alat nya:



➤ Tampilan di Arduino:



➤ Tampilan Dashboardnya

ESP8266 Web Server

Suhu Saat Ini: **1.10** Celcius

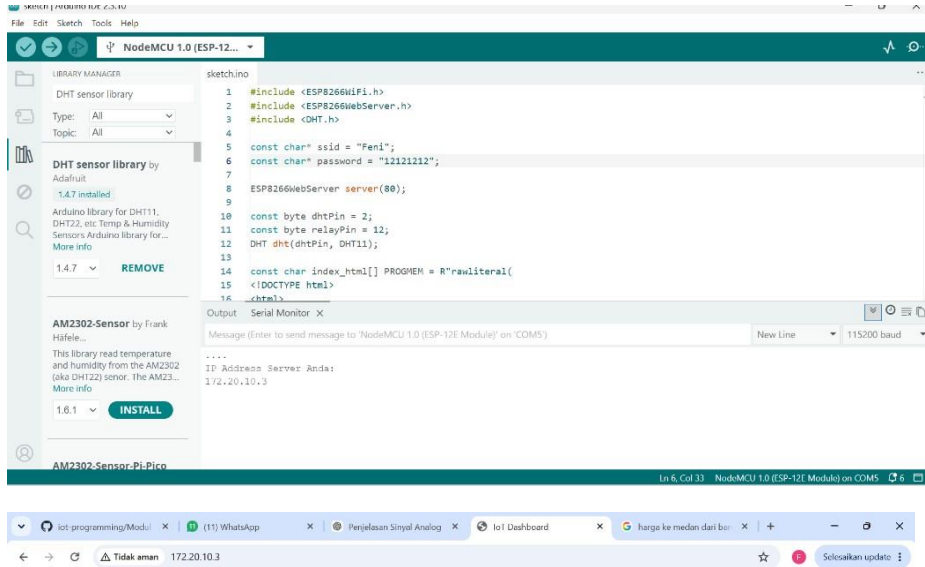
Kendali Relay



Apabila tombol ON di klik maka LED akan menyala dan sebaliknya.

3.5.2 Praktikum 2: NodeMCU sebagai HTTP Client (Meminta Layanan)

Praktikum ini bertujuan Menjadikan NodeMCU sebagai Peminta Layanan (Client) yang dapat bertindak secara mandiri tanpa perlu instruksi dari manusia melalui browser. Tampilan saat di Server menggunakan Praktikum 1 saya sebagai Server:



ESP8266 Web Server

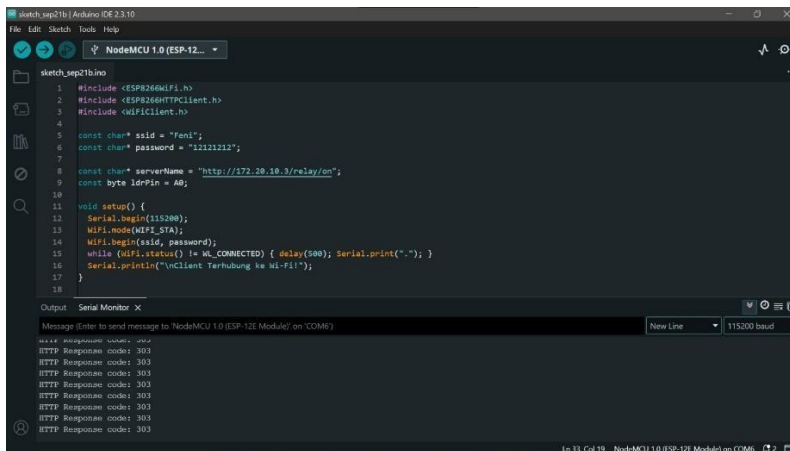
Suhu Saat Ini: 2.20 Celcius

Kendali Relay

ON

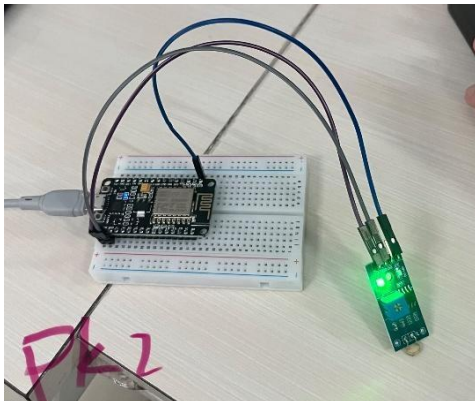
OFF

Tampilan Kode program Saat di Client:

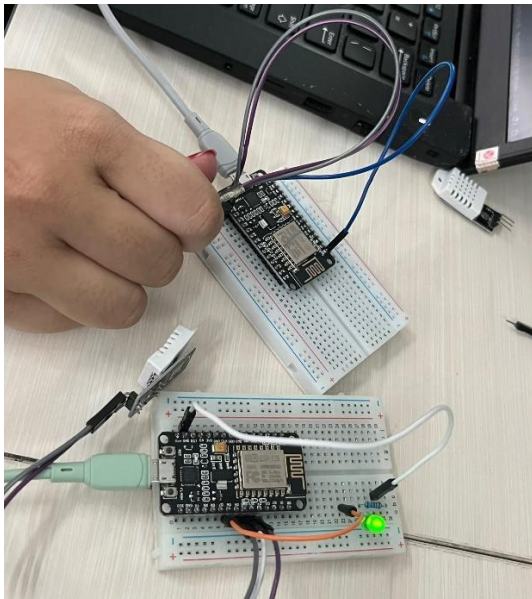


```
1 #include <ESP8266WiFi.h>
2 #include <ESP8266HTTPClient.h>
3 #include <WiFiClient.h>
4
5 const char* ssid = "Feni";
6 const char* password = "12121212";
7
8 const char* serverName = "http://172.28.10.3/relay/on";
9 const byte ledPin = A0;
10
11 void setup() {
12   Serial.begin(115200);
13   WiFi.mode(WIFI_STA);
14   WiFi.begin(ssid, password);
15   while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
16   Serial.println("\nClient terhubung ke Wi-Fi");
17 }
18
19 void loop() {
20   HTTPClient http;
21   http.begin(serverName);
22   int httpCode = http.GET();
23   Serial.println(httpCode);
24   digitalWrite(ledPin, httpCode == 303);
25   delay(1000);
26 }
```

- Hasil Implementasi alat sebelum di tutup tangan atau masih terang pencahayaan nya, LED nya masih OFF :



Hasil Saat LDR ditutup atau penerangan gelap, maka LED akan menyala tanpa menekan tombol OFF di browser client di dashboard server dan LED akan menyala terus sampai tombol OFF ditekan:



➤ HASIL JAWABAN LATIHAN (LATIHAN 1 - 4)

Latihan 1 (Pemahaman Jaringan): Berdasarkan wawasan dari subbab 3.3.1, sebutkan secara logis mengapa kita menggunakan mode jaringan Station (STA) pada kedua praktikum di atas, dan bukan mode Access Point (AP)

JAWABAN : Kita menggunakan mode Station (STA) karena NodeMCU dapat terhubung ke Wi-Fi/Hotspot yang sama dengan HP atau laptop dan NodeMCU lainnya. Dengan begitu, semua perangkat berada dalam satu jaringan dan dapat saling berkomunikasi.

1. Semua perangkat berada dalam satu jaringan

Pada mode STA, NodeMCU Server, NodeMCU Client, HP, dan laptop terhubung ke router atau hotspot yang sama. Contohnya, semua perangkat mendapatkan IP seperti 192.168.43.x. Karena berada dalam jaringan yang sama, HP atau NodeMCU Client dapat mengakses IP dari NodeMCU Server dengan mudah.

2. Lebih praktis dibandingkan mode AP

Jika menggunakan mode Access Point (AP), NodeMCU akan membuat jaringan Wi-Fi sendiri. HP atau NodeMCU Client harus terhubung ke Wi-Fi yang dibuat oleh NodeMCU tersebut. Hal ini kurang praktis jika banyak perangkat yang ingin terhubung.

3. Bisa digunakan untuk akses internet

Mode STA juga memungkinkan NodeMCU tetap terhubung ke jaringan yang memiliki akses internet. Hal ini berguna jika sistem nantinya ingin dikembangkan untuk terhubung ke Cloud IoT, seperti Blynk, ThingSpeak, atau layanan lainnya.

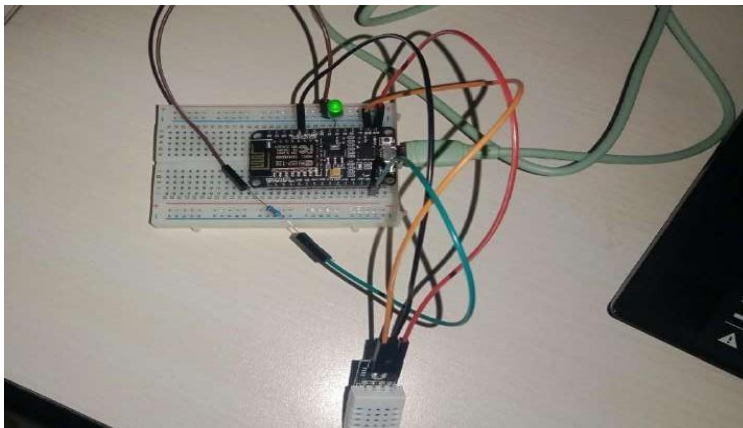
Latihan 2 (Modifikasi Antarmuka & Telemetry): Pada Praktikum 1, tampilan web saat ini baru menyajikan data suhu ruangan dan masih menggunakan dua tombol terpisah (ON dan OFF). Lakukan dua modifikasi berikut pada *Web Server*:

- i. **Menampilkan Kelembapan:** Tambahkan pembacaan kelembapan menggunakan fungsi `dht.readHumidity()`. Sisipkan placeholder `%HUMIDITY%` pada kerangka HTML di dalam tag `<div class="sensor-box">` (contoh: `<p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>`), lalu perbarui fungsi `handleRoot()` menggunakan `html.replace("%HUMIDITY%", ...)` agar nilai kelembapan muncul di halaman web.
- ii. **Tombol Dinamis Tunggal (*Toggle Button*):** Ubah antarmuka agar **hanya menggunakan 1 buah tombol tunggal** yang statusnya adaptif mengikuti kondisi LED saat ini:
 - Jika LED sedang **OFF**, tombol menampilkan tulisan "**NYALAKAN (ON)**" (misal: warna hijau/biru) dan mengarahkan aksi ke `/relay/on`.
 - Jika LED sedang **ON**, tombol otomatis berubah menjadi tulisan "**MATIKAN (OFF)**" (warna merah) dan mengarahkan aksi ke `/relay/off`.

(Petunjuk: Anda dapat menyematkan placeholder baru di HTML seperti %RELAY_BUTTON%, lalu di fungsi handleRoot(), periksa status pin relay/LED menggunakan digitalRead(relayPin) sebelum mengganti placeholder tersebut dengan elemen tautan tombol yang sesuai).

Tuliskan potongan baris kode HTML dan fungsi `handleRoot()` yang Anda ubah!

JAWABAN :



Kode Arduino nya:

```
sketch_sep21a.ino
1  #include <ESP8266WiFi.h>
2  #include <ESP8266WebServer.h>
3  #include <DHT.h>
4
5  const char* ssid = "Fen1";
6  const char* password = "12121212";
7
8  ESP8266WebServer server(80);
9
10 const byte dhtPin = 2;
11 const byte relayPin = 12;
12
13 DHT dht(dhtPin, DHT11);
14
15 const char index_html[] PROGMEM = R"rawliteral(
16 <!DOCTYPE html>
```

```
sketch_sep21a.ino
17 <html>
18 <head>
19   <meta name="viewport" content="width=device-width, initial-scale=1">
20   <title>IoT Dashboard</title>
21
22   <style>
23     body {
24       font-family: Arial;
25       text-align: center;
26       margin-top: 50px;
27     }
28
29     button {
30       padding: 15px 30px;
31       font-size: 20px;
32       border-radius: 8px;
```

```
sketch_sep21a.ino
33     margin: 10px;
34     cursor: pointer;
35   }
36
37   .btn-on {
38     background-color: #4CAF50;
39     color: white;
40     border: none;
41   }
42
43   .btn-off {
44     background-color: #f44336;
45     color: white;
46     border: none;
47   }
48
```

```
sketch_sep21a.ino
49   .sensor-box {
50     font-size: 24px;
51     font-weight: bold;
52   }
53 </style>
54 </head>
55
56 <body>
57
58   <h1>ESP8266 Web Server</h1>
59
60   <div class="sensor-box">
61     <p>Suhu Saat Ini: <strong>%TEMPERATURE%</strong> Celcius</p>
62     <p>Kelembapan: <strong>%HUMIDITY%</strong> %</p>
63   </div>
64
```

```
sketch_sep21a.ino
65   <h2>Kendali Relay</h2>
66
67   %RELAY_BUTTON%
68
69 </body>
70 </html>
71 )rawliteral";
72
73
74 void handleRoot() {
75   String html = index_html;
76
77   // Membaca suhu
78   float t = dht.readTemperature();
79
```

```

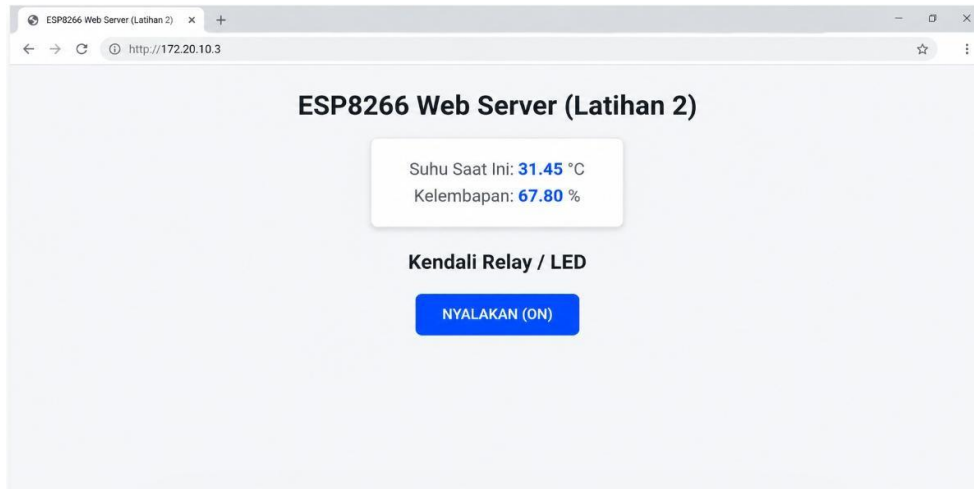
sketch_sep21a.ino
80 // Membaca kelembapan
81 float h = dht.readHumidity();
82
83 // Menampilkan suhu
84 if (isnan(t)) {
85   html.replace("%TEMPERATURE%", "--");
86 } else {
87   html.replace("%TEMPERATURE%", String(t));
88 }
89
90 // Menampilkan kelembapan
91 if (isnan(h)) {
92   html.replace("%HUMIDITY%", "--");
93 } else {
94   html.replace("%HUMIDITY%", String(h));
95 }

sketch_sep21a.ino
90 // Menampilkan kelembapan
91 if (isnan(h)) {
92   html.replace("%HUMIDITY%", "--");
93 } else {
94   html.replace("%HUMIDITY%", String(h));
95 }
96
97 // Membuat tombol dinamis berdasarkan kondisi relay
98 if (digitalRead(relayPin) == LOW) {
99
100   html.replace(
101     "%RELAY_BUTTON%",
102     "<a href=\"/relay/on\"><button class=\"btn-on\">NYALAKAN (ON)</button></a>"
103   );
104
sketch_sep21a.ino
105 } else {
106
107   html.replace(
108     "%RELAY_BUTTON%",
109     "<a href=\"/relay/off\"><button class=\"btn-off\">MATIKAN (OFF)</button></a>"
110   );
111 }
112
113 server.send(200, "text/html", html);
114 }
115
116
117 void handleRelayOn() {
118   digitalWrite(relayPin, HIGH);
119
sketch_sep21a.ino
120 server.sendHeader("Location", "/");
121 server.send(303);
122 }
123
124
125 void handleRelayOff() {
126   digitalWrite(relayPin, LOW);
127
128   server.sendHeader("Location", "/");
129   server.send(303);
130 }
131
132
133 void setup() {
134   Serial.begin(115200);
135
sketch_sep21a.ino
136 pinMode(relayPin, OUTPUT);
137 digitalWrite(relayPin, LOW);
138
139 dht.begin();
140
141 WiFi.mode(WIFI_STA);
142 WiFi.begin(ssid, password);
143
144 Serial.print("Menghubungkan ke Wi-Fi");
145
146 while (WiFi.status() != WL_CONNECTED) {
147   delay(500);
148   Serial.print(".");
149 }
150
151 Serial.println();

sketch_sep21a.ino
152 Serial.println("Wi-Fi Terhubung!");
153 Serial.println("IP Address Server Anda:");
154 Serial.println(WiFi.localIP());
155
156 server.on("/", handleRoot);
157 server.on("/relay/on", handleRelayOn);
158 server.on("/relay/off", handleRelayOff);
159
160 server.begin();
161
162 Serial.println("Web Server dimulai!");
163 }
164
165
166 void loop() {
167   server.handleClient();
168 }

```

Tampilan Webnya :



Latihan 3 (Analisis Kode Client): Pada Praktikum 2, *Serial Monitor Client* mencetak HTTP Response code. Analisislah apa yang akan dicetak oleh *Serial Monitor* Anda jika ternyata NodeMCU *Server* teman Anda mendadak kehilangan daya (*offline*)? (Silakan cabut daya Server dan lihat hasil erornya).

JAWABAN :



- HTTP Response code: 303 Saat Server Nyala

Artinya: Permintaan dari Client berhasil diterima dan diproses oleh Server. Kode 303 (Redirect) muncul karena Server merespons perintah dan mengarahkan kembali ke halaman utama.

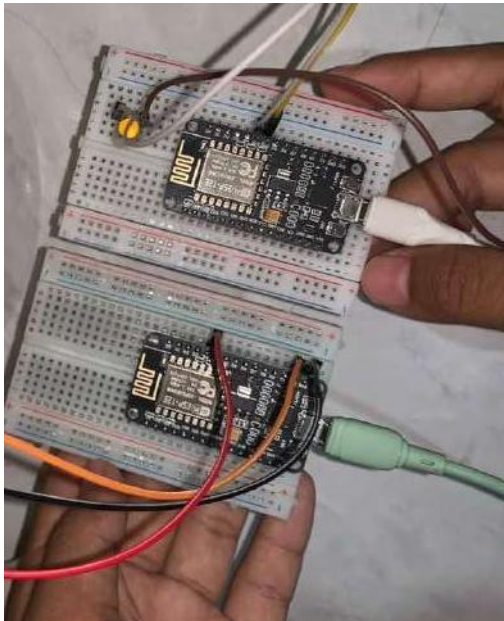
- HTTP Response code: -1 Saat Server dimatikan:

Artinya: Permintaan dari Client gagal terkirim/terhubung karena NodeMCU Server mati, dicabut dayanya, atau kehilangan koneksi.

Sebab: Angka **-1** merupakan kode error dari library HTTP (HTTPC_ERROR_CONNECTION_REFUSED) yang menunjukkan bahwa Client gagal terhubung ke Server pada alamat IP tujuan. Artinya, koneksi ditolak karena Server tidak dapat diakses atau tidak sedang menerima koneksi pada alamat IP tersebut.

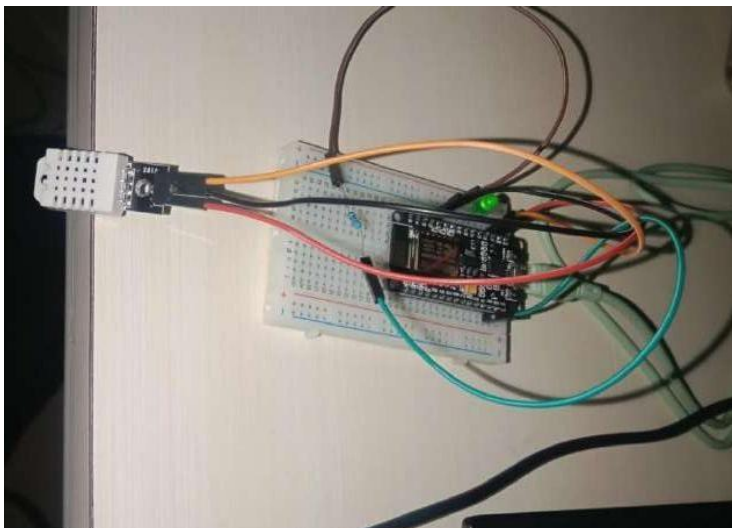
Latihan 4 (Pengembangan Algoritma): Anda ingin mendesain Klien Pendeteksi Suhu. Modifikasi *conditional block* if di Praktikum 2 sehingga *Client* mengirim permintaan (GET Request) **HANYA JIKA** sensor DHT membaca suhu melebihi 35 Celcius (bukan menggunakan LDR). Tulis baris algoritmanya.

JAWABAN : Tampilan dari Arduino di server, IP di ganti karena pengerjaan nya dilakukan di tempat yang berbeda Server adalah board yang ada LED nya, Client nya adalah yang DHT22 untuk membaca suhu, berikut adalah hasil alat nya, agar sensor DHT22 Membaca suhu sampai 35 Celcius kami taruh di magic com, mohon maaf apabila foto yang tidak rapih.

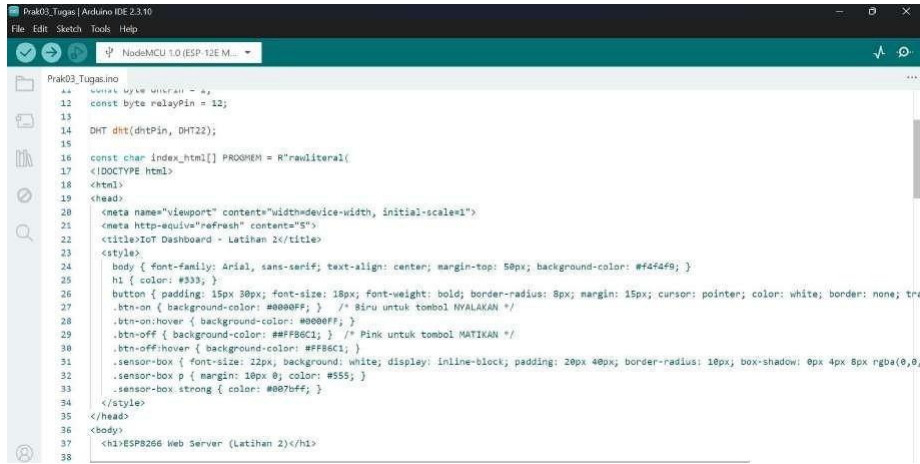


➤ Tugas: 3.7 Tugas: Pengembangan Web Server Cerdas dan Analisis Arsitektur

Berikut adalah hasil dokumentasi dari praktik Tugas:

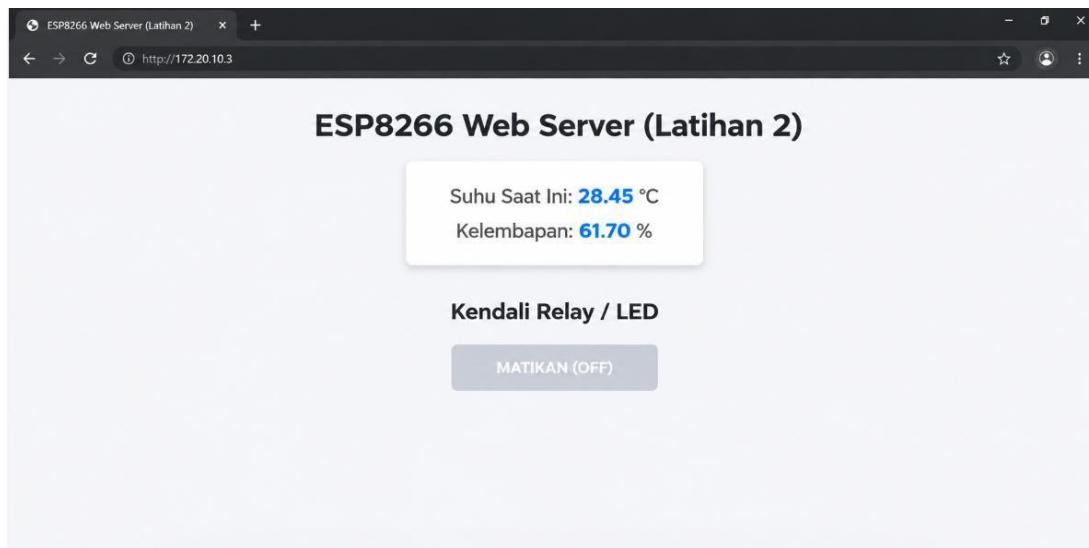


Tampilan kode program di Arduino Uno setelah di sisipkan kode `<meta http-equiv="refresh" content="5">`. :



```
Prak03_Tugas1
11 const byte relayPin = 12;
12
13 DHT dht(dhtPin, DHT22);
14
15 const char index_html[] PROGMEM = R"rawliteral(
16 <!DOCTYPE html>
17 <html>
18 <head>
19 <meta name="viewport" content="width=device-width, initial-scale=1">
20 <meta http-equiv="refresh" content="5">
21 <title>IoT Dashboard - Latihan 2</title>
22 <style>
23 body { font-family: Arial, sans-serif; text-align: center; margin-top: 50px; background-color: #f4f4f9; }
24 h1 { color: #333; }
25 button { padding: 15px 30px; font-size: 18px; font-weight: bold; border-radius: 8px; margin: 15px; cursor: pointer; color: white; border: none; tra
26 .btn-on { background-color: #0080ff; } /* Biru untuk tombol NYALAKAN */
27 .btn-on:hover { background-color: #0066ff; }
28 .btn-off { background-color: #ff66cc; } /* Pink untuk tombol MATIKAN */
29 .btn-off:hover { background-color: #ff3399; }
30 .sensor-box { font-size: 22px; background: white; display: inline-block; padding: 20px 40px; border-radius: 18px; box-shadow: 0px 4px 8px rgba(0,0,
31 .sensor-box p { margin: 10px 0; color: #555; }
32 .sensor-box strong { color: #007bff; }
33 </style>
34 </head>
35 <body>
36 <h1>ESP8266 Web Server (Latihan 2)</h1>
37
38
```

Hasil Tampilan Dashboard akan memberikan instruksi kepada browser yang membuka nya untuk secara otomatis mengirimkan permintaan ulang (HTTP GET Request) ke IP Web Server NodeMCU setiap 5 detik sekali.:



Hasil nya nanti nilai suhu dan kelembapan di browser otomatis berganti seiring kondisi lingkungan tanpa perlu mengeklik tombol refresh di browser.

➤ Kesimpulan

Peran Mode Jaringan ESP8266: NodeMCU ESP8266 menggunakan mode Station (STA) untuk terhubung ke jaringan Wi-Fi yang sama dengan perangkat lain. Dengan begitu, Server, Client, dan Browser dapat saling berkomunikasi melalui jaringan Wi-Fi tanpa menggunakan kabel.

1. Client-Server dan HTTP

NodeMCU sebagai Web Server dapat menerima permintaan HTTP GET, menjalankan perintah seperti / dan /relay/on, serta menampilkan halaman web yang tersimpan di memori NodeMCU.

NodeMCU sebagai HTTP Client dapat mengirim perintah HTTP GET ke Web Server lain secara otomatis berdasarkan kondisi sensor, seperti LDR atau DHT.

2. Tampilan yang Dinamis

Penggunaan satu tombol Toggle membuat tampilan lebih mudah digunakan. Tombol akan berubah sesuai dengan kondisi LED atau relay, sehingga pengguna dapat mengetahui status perangkat secara langsung.

3. Kekurangan Metode Meta Refresh

Penggunaan Meta Refresh dapat memperbarui data suhu dan kelembapan secara otomatis setiap 5 detik. Namun, metode ini memiliki beberapa kekurangan:

- Layar dapat berkedip karena halaman dimuat ulang.
- Menggunakan lebih banyak data dan memori NodeMCU.
- Bisa terjadi masalah jika tombol ditekan bersamaan dengan proses refresh.
- Untuk sistem yang lebih cepat dan real-time, dapat digunakan teknologi seperti AJAX/JSON, WebSocket, atau MQTT.